CDLE · TOOLS

The Container Reference Manual

Docker, Docker Compose and Kubernetes, one task per page

A standalone reference: each part opens with how the tool works, then each page gives the task, the minimal working commands or YAML, and the mistake to avoid.

Docker
Images, containers, Dockerfile, volumes, ports

Compose
compose.yaml, service names, the CDLE stack

Kubernetes
Pods, Deployments, Services, kubectl, Spark on K8s

Troubleshoot
Common errors and a quick reference

Contents

Four parts, one task per page. Each part opens with how the tool works.

01 Docker

5 How Docker works
6 Running containers
7 Writing a Dockerfile
8 Build, tag, push
9 Volumes and mounts
10 Ports and networks

02 Compose

12 How Compose works
13 compose.yaml anatomy
14 Service names as hosts
15 Env and healthchecks
16 Shared settings
17 The Lab 2 stack

03 Kubernetes

19 How Kubernetes works
20 Pods and Deployments
21 Services
22 ConfigMaps and Secrets
23 Debugging with kubectl
24 Spark on Kubernetes

04 Troubleshoot

26 Common errors
27 Quick reference

Conventions: *code blocks show shell commands or YAML, and # starts a comment. Lab examples use the Lab 2 stack on page 17: workers w1–w8, replicas kv1–kv3 and the client.*

Three tools, one stack



Docker and Compose: one machine

Docker builds and runs single containers. Compose starts a whole lab stack from one file. This is what you use every week.

```
docker build  run  ps  logs
docker compose up  down  exec
```

Kubernetes: a cluster

Kubernetes runs the same images on many machines, restarts what fails and scales on demand. Spark can use it as its cluster manager.

```
kubectl apply  get  describe
kubectl logs  exec  delete
```

Rule: *build the image once, then run it unchanged on your laptop, in Compose and on Kubernetes. If it only works on one of them, the difference is in config, not code.*

PART 1 · DOCKER

Docker

01

Package an app and everything it needs into an image, then run that image as a container on any machine.

In this part: How Docker works · Running containers · Dockerfile · Build, tag, push · Volumes · Ports and networks

How Docker works



An image is a template

Read-only and versioned by tag. Images share layers, so ten images built on python:3.11-slim store that base once.

```
docker build  images  tag
docker push  pull  history
```

A container is a running image

It adds a thin writable layer, its own network and its own processes. Many containers can run from one image at once.

```
docker run  ps  logs  exec
docker stop  start  rm
```

Rule: *change the Dockerfile and rebuild; never fix a running container by hand. Changes made inside a container disappear when it is removed.*

Running containers — run, inspect, stop

What a container is

A running process started from an image, with its own filesystem, network and process list. Removing a container never changes its image.

Flags you use every day

- ✓ **-d** — run in the background
- ✓ **--name nb** — a fixed name, not a random one
- ✓ **-p 8888:8888** — publish a port, host:container
- ✓ **--rm** — delete the container when it exits
- ✓ **-e KEY=value** — set an environment variable

Start, inspect and clean up

```
docker run -it --rm python:3.11-slim python

docker run -d --name nb -p 8888:8888 \
  quay.io/jupyter/pyspark-notebook

docker ps                    # running containers
docker ps -a                 # include stopped ones
docker logs -f nb            # follow the output
docker exec -it nb bash      # shell inside it
docker stop nb               # stop gracefully
docker rm nb                 # delete the container
docker system prune          # remove unused data
```

Watch out: *a container stops when its main process exits. If docker ps shows nothing, run docker ps -a and docker logs <name> to see why it ended.*

Writing a Dockerfile — the build recipe

How a build works

Each instruction adds a cached layer. Docker rebuilds from the first one that changed, so put what changes least at the top.

Instructions

- ✓ **FROM —** the base image to start from
- ✓ **RUN —** a command run once, at build time
- ✓ **COPY —** files from the build folder
- ✓ **ENV / WORKDIR —** variables and the working folder
- ✓ **CMD —** the default command at run time

Mount or copy the code?

- ✓ **Mount —** for labs: edit a script and rerun, no rebuild
- ✓ **Copy —** for jobs you ship: the image runs on its own

Lab 2 image, and a shipped job

```
# Lab 2: tools in the image, code mounted
FROM python:3.12-slim
RUN pip install --no-cache-dir duckdb==1.4.5
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /lab
# compose.yaml mounts the scripts: .:/lab:ro

# a shipped job: code copied in
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
CMD ["python", "src/job.py"]
```

Watch out: *pin versions (duckdb==1.4.5) so every student gets the same build. Copy dependencies before code, or every edit reinstalls every package. Never COPY .env files or secrets into an image.*

Build, tag, push — images and registries

Image names

registry/repository:tag, for example ghcr.io/cdle/job:1.0. With no registry Docker assumes Docker Hub; with no tag it assumes latest.

Commands

- ✓ **docker build -t** — build an image from a folder
- ✓ **docker images** — list local images
- ✓ **docker tag** — give an image another name
- ✓ **docker push / pull** — upload to or download from a registry
- ✓ **docker history** — layers and their sizes

Build and publish a version

```
# build from the Dockerfile in this folder
docker build -t cdle/job:1.0 .

# rebuild from scratch, ignoring the cache
docker build --no-cache -t cdle/job:1.0 .

# name it for a registry, then push
docker tag cdle/job:1.0 ghcr.io/cdle/job:1.0
docker login ghcr.io
docker push ghcr.io/cdle/job:1.0

# on any other machine
docker pull ghcr.io/cdle/job:1.0
```

Watch out: *latest is only a default tag, not the newest build. Tag every build with a version so Compose and Kubernetes run the image you tested. On an Apple Silicon Mac, add --platform linux/amd64 for x86 servers.*

Volumes and mounts — keep your data

Why mount anything

A container's own files are deleted with the container. Data you want to keep, such as databases, notebooks or lake files, goes on a mount.

Three kinds

- ✓ **Named volume** — managed by Docker; for databases and brokers
- ✓ **Bind mount** — a host folder; for code and notebooks you edit
- ✓ **tmpfs** — memory only; for scratch files
- ✓ **Syntax** — -v name:/path is a volume, -v ./dir:/path a folder

A named volume and a bind mount

```
# named volume: survives docker rm
docker volume create pgdata
docker run -d --name db \
  -e POSTGRES_PASSWORD=cdle \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# bind mount: edits on the host show up live
docker run -it --rm -p 8888:8888 \
  -v "$PWD/notebooks:/home/jovyan/work" \
  quay.io/jupyter/pyspark-notebook

docker volume ls
docker volume rm pgdata     # deletes the data
```

Watch out: *docker compose down keeps named volumes, but down -v deletes them. On Linux, files a container writes to a bind mount belong to the container's user, which may not be you.*

Ports and networks — reach a container

Two directions

Published ports connect your machine to a container. Networks connect containers to each other, by name.

Rules

- ✓ **-p 5433:5432** — host port 5433 to container port 5432
- ✓ **localhost** — inside a container, means that container
- ✓ **User network** — containers find each other by name
- ✓ **Default bridge** — no name lookup; create a network instead

Two containers that talk by name

```
docker network create lab

docker run -d --name db --network lab \
  -e POSTGRES_PASSWORD=cdle postgres:16

# a second container reaches it as host "db"
docker run -it --rm --network lab \
  postgres:16 psql -h db -U postgres

# your laptop reaches it through a port
docker run -d --name db2 -p 5433:5432 \
  -e POSTGRES_PASSWORD=cdle postgres:16
docker port db2              # 5432 -> 0.0.0.0:5433
docker network inspect lab
```

Watch out: *if the host port is taken, change the left number, not the right one. A server that listens on 127.0.0.1 inside the container cannot be reached from outside; bind it to 0.0.0.0.*

PART 2 · DOCKER COMPOSE

Compose

02

Describe a whole multi-container stack in one YAML file and start it with one command.

In this part: How Compose works · compose.yaml anatomy · Service names as hosts · Env and healthchecks · Shared settings · The Lab 2 stack

How Docker Compose works



One file describes the stack

Instead of five docker run commands with long flags, you commit one compose.yaml, and everyone starts the same stack.

```
services:  image:  build:
ports:  volumes:  environment:
```

Commands act on the whole stack

Run them in the folder that holds compose.yaml. Add a service name to act on that service only.

```
docker compose up -d  down  ps
docker compose logs  exec  build
```

Rule: *inside the stack, services reach each other by service name on the container port. Published ports are only for your browser and tools on the host.*

compose.yaml anatomy — one file, the whole stack

Top-level keys

- ✓ **services** — one entry per container
- ✓ **volumes** — named volumes the services use
- ✓ **networks** — optional; one default network is made

Per-service keys

- ✓ **image / build** — pull an image or build a folder
- ✓ **ports** — publish to the host, host:container
- ✓ **volumes** — mounts, like docker run -v
- ✓ **environment** — variables inside the container
- ✓ **depends_on** — start order between services

compose.yaml

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: cdle
    volumes:
      - pgdata:/var/lib/postgresql/data
  app:
    build: ./app              # uses ./app/Dockerfile
    ports:
      - "8080:8080"
    depends_on: [db]
volumes:
  pgdata:
```

Watch out: *YAML indentation is the structure: use spaces, never tabs. Quote port mappings like "8080:8080"; unquoted, some values are read as numbers. The old version: key is no longer needed.*

Service names are hostnames — not localhost

Compose puts every service on one network and registers its name in DNS. Containers use the name and the container port; your laptop uses localhost and the published port.

From another container: the service name

```
# inside the client container (Lab 2)
WORKERS = [f"w{i}:8100" for i in range(1, 9)]
REPLICAS = ["kv1", "kv2", "kv3"]
# Spark labs: spark://spark-master:7077

# check the name lookup
docker compose exec client \
  getent hosts w1 kv2
```

From your laptop: localhost and the published port

```
# ports published in compose.yaml
#   127.0.0.1:8101:8100   w1
#   ...
#   127.0.0.1:8108:8100   w8

docker compose port w3 8100
# 127.0.0.1:8103

# w1:8100 does not resolve here;
# use localhost:8101 ... :8108
# kv1..kv3 publish no ports
```

Watch out: *inside a container, localhost:8100 is that container itself, so the connection is refused. Use the service name. Ports bound to 127.0.0.1 answer on this machine only (page 16).*

Env and healthchecks — configure, then wait

Configuration

✓ **environment:** — values written in the file

✓ **env_file:** — loads KEY=value lines from a file

✓ **.env** — fills ${VAR} in compose.yaml; never commit it

Start order

depends_on alone waits for the container to start, not for the service inside it to be ready. Add a healthcheck and wait for service_healthy.

Wait until the database answers

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 5s
      retries: 10
  app:
    build: ./app
    env_file: app.env
    depends_on:
      db:
        condition: service_healthy
# .env, same folder:  DB_PASSWORD=cdle
```

Watch out: *${VAR} is filled from .env or your shell when Compose reads the file, not inside the container. Run docker compose config to see the final file with every value filled in.*

Shared settings — anchors, limits, binds

Write a setting once

- ✓ **x-name: —** a block Compose ignores; holds shared keys
- ✓ **&node / *node —** a YAML anchor and a reference to it
- ✓ **<<: *node —** merge those keys; keys below override

Keys the Lab 2 file uses

- ✓ **cpus / mem_limit —** cap a container; past the memory cap it is killed (code 137)
- ✓ **container_name —** a fixed name, such as w1
- ✓ **init: true —** forwards stop signals, reaps child processes
- ✓ **stop_grace_period —** wait before a hard kill on stop

Check the merge

- ✓ **docker compose config —** prints every service as Compose sees it

From the Lab 2 compose.yaml

```
x-node: &node
  image: cdle-lab2
  init: true
  stop_grace_period: 1s
x-worker: &worker
  <<: *node                # every key of x-node
  command: python worker.py
  cpus: 1.0
  mem_limit: 1g
services:
  w1:
    <<: *worker
    container_name: w1
    ports: ["${LAB2_BIND:-127.0.0.1}:8101:8100"]

# unset: 127.0.0.1, this machine only
LAB2_BIND=0.0.0.0 docker compose up -d
```

Watch out: *${VAR:-default} takes the default when VAR is unset or empty, so the Lab 2 ports stay on 127.0.0.1 unless you set LAB2_BIND.*

The Lab 2 stack — workers, replicas, client

Twelve containers, one image

Compose builds cdle-lab2 once from the lab's Dockerfile; each service runs it with its own command and limits.

- ✓ **w1–w8 —** query workers; 1 CPU, 1 GB each
- ✓ **kv1–kv3 —** key-value replicas; 0.5 CPU, 128 MB
- ✓ **client —** runs partition.py, coordinator.py, kv_client.py

Every lab session

- ✓ **docker compose up -d --build —** build and start
- ✓ **docker compose exec client bash —** a shell in the client
- ✓ **docker compose down —** stop; the data volume stays

Two machines (optional)

- ✓ **LAB2_BIND=0.0.0.0 —** publish worker ports to the network; see Appendix B

compose.yaml, abridged

```yaml
x-node: &node                  # all services
  image: cdle-lab2
  build: .
  volumes: [".:/lab:ro", "data:/data:ro"]
x-worker: &worker
  <<: *node
  command: python worker.py
  cpus: 1.0
  mem_limit: 1g
services:
  w1:                          # w2..w8 alike
    <<: *worker
    ports: ["${LAB2_BIND:-127.0.0.1}:8101:8100"]
  kv1: {<<: *kv}               # kv2, kv3
  client:
    <<: *node
    command: sleep infinity    # you exec in
volumes:
  data:
```

Watch out: */lab is mounted read-only, so scripts write to /data, which only the client can write. The data sits in a Docker volume, not a host folder: shared folders are slow on macOS and Windows.*

PART 3 · KUBERNETES

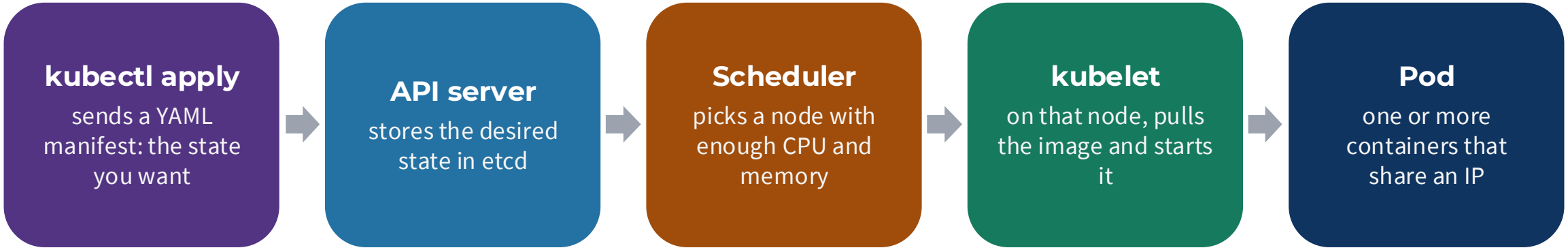
Kubernetes

03

Run the same images across a cluster: Kubernetes schedules them, restarts what fails and scales on demand.

In this part: How Kubernetes works · Pods and Deployments · Services · ConfigMaps and Secrets · Debugging with kubectl · Spark on Kubernetes

How Kubernetes works



Desired state, not commands

You declare replicas: 3. Controllers compare that with what is running and fix the difference, restarting or moving Pods.

```
kubectl apply -f app.yaml
```

```
kubectl get  describe  delete
```

Objects you will use

A Pod runs containers, a Deployment keeps N Pods running, a Service gives them a stable name, ConfigMaps and Secrets hold settings.

```
pod  deployment  service
```

```
configmap  secret  namespace
```

Rule: *Pods are disposable: each restart gets a new name and IP. Never point at a Pod; point at a Service.*

Pods and Deployments — keep N copies running

Pod

The smallest unit: one or more containers sharing an IP and volumes. You rarely create Pods yourself; a Deployment does it.

Deployment

- ✓ **replicas** — how many Pods to keep running
- ✓ **selector** — which Pods it owns, by label
- ✓ **template** — the Pod it creates
- ✓ **rolling update** — a new image replaces Pods a few at a time

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: {app: api}
  template:
    metadata:
      labels: {app: api}
    spec:
      containers:
        - name: api
          image: ghcr.io/cdle/api:1.0
          ports: [{containerPort: 8080}]
```

Watch out: *the selector must match the template labels exactly, or the apply is rejected. After editing, kubectl apply -f again; kubectl rollout undo deployment/api goes back one version.*

Services — a stable name for changing Pods

What a Service gives you

A fixed name and IP in front of whichever Pods match its label. Other Pods in the namespace call http://api:80.

Types

- ✓ **ClusterIP** — inside the cluster only (default)
- ✓ **NodePort** — a port on every node, 30000–32767
- ✓ **LoadBalancer** — an external IP from the cloud
- ✓ **port-forward** — a tunnel from your laptop, for debugging

service.yaml, then reach it

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector: {app: api}     # Pods with this label
  ports:
    - port: 80             # the Service port
      targetPort: 8080     # the container port

kubectl apply -f service.yaml
kubectl get endpoints api  # Pods behind it
kubectl port-forward svc/api 8080:80
```

Watch out: *if kubectl get endpoints shows no addresses, the selector matches no Pod labels. From another namespace, use the full name api.<namespace>.svc.cluster.local.*

ConfigMaps and Secrets — config outside the image

Why separate them

The same image runs in every environment; only its ConfigMap and Secret change between lab, test and production.

Two objects, two ways in

- ✓ **ConfigMap** — plain settings: URLs, flags, files
- ✓ **Secret** — passwords and tokens, kept apart
- ✓ **envFrom** — every key becomes an env variable
- ✓ **volume** — every key becomes a file

Create them, then use them

```
kubectl create configmap api-config \
  --from-literal=KAFKA=kafka:9092
kubectl create secret generic db-cred \
  --from-literal=PASSWORD=cdle

# in the Deployment's container spec
envFrom:
  - configMapRef: {name: api-config}
  - secretRef: {name: db-cred}

kubectl get configmap api-config -o yaml
```

Watch out: *Secret values are base64-encoded, not encrypted; anyone who can read the Secret can decode it. Pods read env variables only at start: run kubectl rollout restart deployment/api after a change.*

Debugging with kubectl — status, events, logs

Read in this order

✓ **get** — is it Running? How many restarts?
✓ **describe** — the Events at the bottom say why
✓ **logs** — what the app printed; --previous for the crash
✓ **exec** — a shell inside, to test from there

Where you are

✓ **-n lab** — the namespace to act in
✓ **config get-contexts** — which cluster kubectl talks to

From status to cause

```
kubectl get pods -n lab -o wide
kubectl describe pod api-7d9f-x2k -n lab
kubectl logs api-7d9f-x2k -n lab
kubectl logs api-7d9f-x2k --previous
kubectl logs -f deploy/api        # follow
kubectl exec -it api-7d9f-x2k -- sh

kubectl get events --sort-by=.lastTimestamp
kubectl top pods         # needs metrics-server

kubectl config get-contexts
kubectl config set-context --current \
  --namespace=lab
```

Watch out: *a climbing RESTARTS count means the container keeps crashing; the reason is in logs --previous, not the current logs. Forget -n and you see the default namespace, which looks empty.*

Spark on Kubernetes — driver and executors as Pods

How it runs

spark-submit calls the API server. Kubernetes starts a driver Pod, the driver asks for executor Pods, and they are removed when the job ends.

What you need

- ✓ **An image** — Spark plus your job, e.g. on apache/spark:3.5.1
- ✓ **Pod rights** — the namespace's service account may create Pods
- ✓ **k8s:// master** — the API server address
- ✓ **local:// paths** — files inside the image

Follow the job

- ✓ **spark-role=driver** — the label on the driver Pod

Submit a job in cluster mode

```
# let the default service account create Pods
kubectl create rolebinding spark-edit -n lab \
  --clusterrole=edit --serviceaccount=lab:default

# spark-k8s.conf
#  spark.kubernetes.namespace        lab
#  spark.kubernetes.container.image  cdle/job:1.0
#  spark.executor.instances          3

spark-submit --master k8s://https://<api>:6443 \
  --deploy-mode cluster --name clicks-report \
  --properties-file spark-k8s.conf \
  local:///opt/spark/work-dir/job.py
kubectl get pods -n lab -l spark-role=executor
```

Watch out: *in cluster mode the driver runs in a Pod, so read its output with kubectl logs <driver-pod>. It cannot see files on your laptop: put the job in the image or read data from S3 or HDFS.*

PART 4 · TROUBLESHOOT

Troubleshoot

04

When something fails, read the status and the logs first, then match the error to its usual cause.

In this part: Common errors · Quick reference

Common errors — what they mean and the fix

Error or symptom	Usual cause	Fix
port is already allocated	Another container or app holds that host port	Change the host side: -p 5433:5432
Exited (137), OOMKilled	The container ran out of memory	Raise mem_limit in compose.yaml (Lab 2 kv: 128m) or resources.limits
Connection refused on localhost	Inside a container, localhost is the container itself	Use the service name, such as w1:8100 (page 14)
ImagePullBackOff	Wrong image name or tag, or a private registry	Test with docker pull; add an imagePullSecret
CrashLoopBackOff	The app exits right after it starts	kubectl logs <pod> --previous (page 23)
Pod stuck in Pending	No node has the CPU or memory requested	Read Events in kubectl describe; lower requests

Reading a failure: *in Docker, run docker ps -a and docker logs; in Kubernetes, kubectl describe and kubectl logs. The first error line names the cause; the rest is fallout.*

Quick reference

Docker
docker …

```
run -d -p 80:80 <img>
ps -a
logs -f <c>
exec -it <c> sh
build -t img:tag .
push / pull <img>
volume ls
system prune
```

Docker Compose
docker compose …

```
up -d
down   (-v drops data)
ps
logs -f <svc>
exec <svc> sh
build
config
up -d --scale <svc>=3
```

kubectl: inspect
kubectl … changes nothing

```
get pods -o wide
get deploy,svc
describe pod <p>
logs <p> --previous
get events
top pods
config get-contexts
```

kubectl: change
kubectl … apply and roll

```
apply -f app.yaml
delete -f app.yaml
scale <d> --replicas=3
rollout restart <d>
rollout undo <d>
port-forward svc/x 80
exec -it <p> -- sh
```

Remember: *containers are disposable. Keep code in the image, config in env variables or ConfigMaps, and data on volumes, and you can delete and recreate anything.*